Fundamentals of API Development

Breakout Lab 2.1: Designing a Non-Breaking Version Strategy

Day 3

KavinSchool

Lab Overview

Goal: Classify proposed changes as breaking or non-breaking using repeatable code, choose and justify a versioning strategy, and write a deprecation plan for the version being replaced.

- **Format:** Individual or pair work
- **Tools:** IntelliJ, the companion repo (API does not need to be running)

> **NOTE**
>
> You just watched `DemoNonBreakingChangeChecklist` classify six proposed changes as breaking or non-breaking. This lab asks you to build that classifier yourself, then use it to decide when to bump a version.

What's in the Companion Repo (1/2)

Open `day3/lab2_1_designing_a_non_breaking_version_strategy/` in the companion repo.

Path	What it is
`start/ChangeClassifier.java`	The classifier you complete. The field-rename check is the worked example; five TODOs are yours.
`start/VERSION-STRATEGY.md`	The strategy worksheet: choose a strategy, justify it, and write a deprecation plan.
`solution/`	Completed classifier and filled-in worksheet. Open after you have tried.

What's in the Companion Repo (2/2)

NO API NEEDED

This lab works against the endpoints' documented shape from `GreetingController` and `HealthController`, which you can read directly.

Step 1: Implement the Checklist

The Worked Example

Open `start/ChangeClassifier.java`. Read `checkFieldRename` first, it is the worked example. The five TODOs follow the same shape.

Run it now and watch the output:

```
mvn exec:java -pl day3/lab2_1_designing_a_non_breaking_version_strategy/start \
  -Dexec.mainClass="com.kavinschool.api.ChangeClassifier"
```

It reports 0% breaking changes. That score is incomplete. Making it accurate is your job.

The Five TODOs

TODO	Rule	Watch out for
1	Field addition (non-breaking)	Adding a new optional response field does not break existing consumers
2	Field removal (breaking)	Removing a field consumers depend on requires a new version
3	Endpoint addition (non-breaking)	A new path is additive, nothing changes for existing callers
4	HTTP method change (breaking)	Changing `GET` to `POST` on the same path breaks every existing client
5	Parameter tightening (breaking)	Making an optional parameter required rejects previously-valid requests

Then Add Your Own

After implementing the five TODOs, complete **TODO 6**: add two of your own proposed changes to the `PROPOSED_CHANGES` list. Classify them. At least one should be breaking, at least one non-breaking.

> **PROVE IT WORKS**
>
> Add a deliberately wrong classification, like calling a field removal non-breaking, and confirm your check fires. Remove it afterwards.

Step 2: Choose a Strategy

Fill In the Worksheet (1/2)

Open `start/VERSION-STRATEGY.md`. Answer three questions:

1. **Which versioning strategy?** Choose from URI path (`/api/v1/...`), custom header (`X-API-Version`), or content negotiation (`Accept: vnd.api.v2+json`). Justify your choice against the API's audience and the tradeoffs from the session.
2. **Where does the version live?** In the path, in the header, in the media type, where exactly?
3. **Deprecation plan.** Write a three-phase deprecation plan for the version being replaced: soft deprecation (warning), hard deprecation (rate-limited), and final shutdown (published sunset date).

Fill In the Worksheet (2/2)

THE REAL SKILL

Anyone can bump a version number. Deciding when to bump it, and how to retire the old one, is what this session is teaching.

Lab Completion Checklist

Checklist

- [ ] All five TODO classification rules implemented and tested
- [ ] TODO 6 completed: two proposed changes of your own, classified correctly
- [ ] At least one deliberately wrong classification tested and caught
- [ ] Versioning strategy chosen with a written justification
- [ ] Three-phase deprecation plan written: soft → hard → shutdown
- [ ] Version location specified (path, header, or media type)

DONE?

Compare your deprecation plan with a neighbor. The three phases should match, but the timeline (how long between soft and hard deprecation) will vary, and that's the useful debate.

Lab 2.1 Complete

Keep your classifier and strategy, you'll use the same versioning discipline in the capstone

KavinSchool